# THE HP PROTEIN-FOLDING PROBLEM: COMPARING CONSTRAINT PROGRAMMING AND REINFORCEMENT LEARNING

A PREPRINT

**Pranav Ramanathan**
School of Mathematical Sciences
Queen Mary University of London
p.ramanathan@se23.qmul.ac.uk

**Thomas Prellberg**
School of Mathematical Sciences
Queen Mary University of London
t.prellberg@qmul.ac.uk

March 15, 2026

**ABSTRACT**

This paper presents an analysis of solving the 3D HP protein-folding problem using CP-SAT and compares its performance with Deep Q-Learning (DQN). The HP protein-folding problem is a classic NP-complete problem in the field of computational biology and combinatorial optimisation that has been studied extensively. CP-SAT is an open-source constraint programming solver that systematically explores the solution space through constraint satisfaction. DQN is a reinforcement learning algorithm that learns to fold proteins by interacting with the environment and optimizing a policy through trial-and-error. This paper discusses the main components of the CP-SAT approach, such as constraint formulation, contact maximization, and symmetry breaking, and explains how it is used to solve the HP protein-folding problem. The Deep Q-Learning approach, including the Q-network architecture, experience replay, and reward structure for maximizing H-H contacts, is also discussed. The experimental results show that CP-SAT outperforms learning-based methods for the benchmark sequences tested, matching the best known energy levels for the given data.

***Keywords*** HP model · protein folding · constraint programming · CP-SAT · reinforcement learning · deep Q-learning · lattice models

## 1 Introduction

Proteins are biological macromolecules that fold into three-dimensional structures, which determine their unique functions. These functions range from catalysing biochemical reactions to defending against pathogens. Because a protein's activity is strongly shaped by its conformation, characterising folding behaviour is often essential for understanding how proteins interact with other molecules. In medicine, this can inform drug discovery by supporting the design of compounds intended to stabilise or otherwise modulate specific protein structures.

This brings us to a central challenge in molecular biology: predicting a protein's three-dimensional conformation from its amino acid sequence. That task matters for drug design, disease research, and protein engineering. Levinthal's Paradox, introduced by Cyrus Levinthal in 1969 [1], is often cited to show why an exhaustive search over all possible conformations is not a realistic explanation for how proteins fold. The key observation is that, even under conservative assumptions, the conformational search space becomes astronomically large.

Haemoglobin provides a useful illustration. The oxygen-carrying protein in red blood cells has a beta subunit containing 146 amino acids [2], and each position in the chain is drawn from 20 standard amino acid types [3]. What makes folding difficult is how quickly the space of plausible conformations expands as the chain grows. If each rotatable bond is assumed to have $k$ stable conformations and $n$ is the number of amino acids, a rough upper-bound count gives on the order of $k^{n-1}$ possible configurations. For a beta subunit that gives 145 rotatable bonds, taking $k$ as roughly 3 yields $3^{145}$, or about $10^{69}$ configurations. Even at an unrealistically fast sampling rate of one configuration per picosecond, enumerating the full set would take on the order of $10^{49}$ years. Yet haemoglobin folds spontaneously on millisecond-to-second timescales [4]. The gap between these numbers is the intuition behind Levinthal's Paradox,

and it is one reason folding is typically approached through models and algorithms that exploit structure rather than brute-force enumeration.

To make the problem more tractable, Dill introduced the Hydrophobic-Polar (HP) lattice model in 1985 [5]. The model collapses the twenty amino acid types into two classes: hydrophobic residues (H), which tend to avoid water, and polar residues (P), which interact more favourably with it. A sequence is then embedded as a self-avoiding walk on a lattice, with each residue occupying a distinct site and successive residues constrained to lie on adjacent lattice points [6]. Within this discrete setting, the objective is driven by hydrophobic contacts. The energy is defined by the number of non-consecutive H-H contacts, i.e., pairs of hydrophobic residues that are neighbours on the lattice but not adjacent along the chain. In effect, the optimisation target is to find an embedding that promotes hydrophobic clustering while respecting the self-avoidance and adjacency constraints.

Even under this simplified lattice formulation, HP folding remains computationally demanding. Crescenzi et al. proved in 1998 that the optimisation problem is NP-complete [7], so one should not expect a method that scales efficiently across all instances as the sequence length increases. The practical consequences show up fairly quickly. By the time sequences are in the 30 to 50 residue range, the set of valid self-avoiding conformations is already large enough that exhaustive enumeration stops being a useful point of comparison. That is part of the reason the HP model is used so often in combinatorial optimisation research: it is clean to specify, hard to solve, and easy to evaluate consistently across algorithms.

Given that hardness, much of the earlier computational literature relied on heuristics rather than exact optimisation. Simulated annealing became a common choice from the 1990s onward because it provides a straightforward mechanism for escaping local minima by occasionally accepting unfavourable moves early in the search [8]. Genetic algorithms offered a different style of exploration, maintaining a population of candidate folds and gradually improving it through recombination and mutation operators [9]. Tabu search was also explored, discouraging cycling by remembering recent states. Ant colony optimisation took yet another approach, biasing construction using information carried over from successful searches [10, 11].

Constraint programming (CP) offers an alternative to heuristic search for HP folding. Chain connectivity, self-avoidance, and contact formation can all be encoded directly as folding constraints and then pruned through propagation [12]. Earlier CP tools, such as CPSP-tools, successfully tackled HP folding using specialised H-core databases and constraint satisfaction techniques [13]. Google OR-Tools CP-SAT extends this approach by combining it with SAT-style clause learning and linear programming relaxation [14]. This can provide bounds on solution quality and, in favourable cases, certificates of optimality [15]. Large Neighbourhood Search is also supported, allowing solutions to be improved iteratively [16]. Modern CP-SAT solvers, like CP-SAT, have not been applied to HP folding, as earlier CP tools were [13], making this a fascinating venue worth revisiting.

While constraint programming guarantees optimality when feasible, its exponential scaling limits practical applicability to shorter sequences. Reinforcement learning has recently emerged as an alternative approach for HP folding. Espitia et al. (2024) applied Deep Q-learning to the three-dimensional HP model [17]. Folding was treated as a sequential decision problem, where agents can learn folding strategies through trial-and-error without explicitly encoding the problem constraints [18]. Whether this reinforcement learning method can compete with exact solver approaches like CP-SAT remains unknown.

This study presents a rigorous comparison of constraint programming and reinforcement learning for the 3D HP protein folding problem. We implement CP-SAT using Google OR-Tools with Large Neighbourhood Search and compare its performance against the Deep Q-Learning approach from Espitia et al. (2024) [17], which we adapt and evaluate on our benchmark sequences.

**Paper structure as follows:**

- Section 2 describes the implementation and training procedures for each method.
- Section 3 presents comparative results and scaling behaviour.
- Section 4 discusses algorithmic trade-offs, failure modes, and future research directions.

## 2 Methodology

### 2.1 Problem Formulation

The HP protein folding problem on a three-dimensional lattice is defined as follows. A protein sequence is represented as a string:

$$S = s_1 s_2 \dots s_n, \tag{1}$$

where each residue $s_i$ is classified as either hydrophobic (H) or polar (P). A conformation $C$ is a self-avoiding walk on the three-dimensional integer lattice $\mathbb{Z}^3$, meaning each residue is placed at a distinct position:

$$c_i = (x_i, y_i, z_i) \in \mathbb{Z}^3, \tag{2}$$

and no two residues occupy the same lattice point. The adjacency constraint requires that consecutive residues along the chain are placed at Manhattan distance 1:

$$||c_i - c_{i+1}||_1 = 1 \quad \forall i \in \{1, \ldots, n-1\}. \tag{3}$$

where $||v||_1 = |x| + |y| + |z|$ for a vector $v = (x, y, z)$.

The energy of a conformation is determined by the number of non-consecutive H-H contacts. A contact occurs when two hydrophobic residues are neighbours on the lattice but not adjacent along the sequence. Formally, the energy function is defined as

$$E(C) = -\left|\{(i,j) : s_i = s_j = \mathrm{H},\ |i-j| > 1,\ ||c_i - c_j||_1 = 1\}\right|, \tag{4}$$

where the set counts all pairs of non-consecutive H-residues that are lattice-adjacent. The objective is to minimise $E(C)$, which is equivalent to maximising the number of favourable hydrophobic contacts. This formulation reflects the simplified physical principle that hydrophobic residues prefer to cluster together, shielding themselves from the surrounding aqueous environment. An illustration of this energy function is provided in Figure 1.

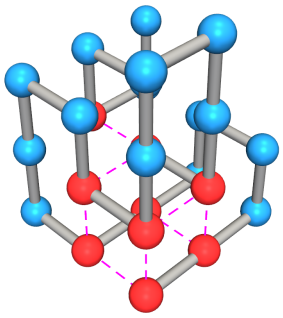


Figure 1: Illustration of the 3D HP lattice model showing a short self-avoiding walk. Hydrophobic residues (H) are shown in red, polar residues (P) in blue. Dotted magenta lines indicate valid non-consecutive H-H contacts that contribute to the energy function; this conformation has energy $-9\epsilon_{HH}$.

We evaluate both methods on a set of benchmark sequences drawn from the literature. Table 1 lists the sequences used in this study, identified by their standard names (e.g., 3d1, 3d2, ..., 3d8). The sequences span a range of lengths from $n = 20$ to $n = 58$ residues and include both their H/P patterns and best-known energy values from prior work [17].

### 2.2 CP-SAT Approach

This section describes our constraint programming implementation using Google OR-Tools CP-SAT solver [15, 14]. We formulate the 3D HP folding problem as a constraint satisfaction problem, encoding the self-avoiding walk and contact constraints algebraically to maximise H-H contacts.

#### 2.2.1 Decision Variables

Our CP-SAT formulation uses one-hot encoding to represent where residues sit on the 3D cubic lattice. In one-hot encoding, each position is indicated by a binary vector where exactly one element is set to 1 (indicating the residue occupies that position) and all others are 0. For each residue $i \in \{0, 1, \ldots, n-1\}$ and each cell $c = (x, y, z)$ in the grid:

$$\mathcal{G} = \{(x, y, z) : x, y, z \in \{0, 1, \ldots, L-1\}\}, \tag{5}$$

we create a binary decision variable:

$$x_{i,c} \in \{0, 1\}, \tag{6}$$

Table 1: Benchmark HP sequences used in this study. Best-known energies are taken from [17].

| Seq. | Length | Sequence | Best-known Energy |
|---|---|---|---|
| 3d1 | 20 | $(HP)^2PH(HP)^2(PH)^2HP(PH)^2$ | $-11$ |
| 3d2 | 24 | $H^2P^2(HP^2)^6H^2$ | $-13$ |
| 3d3 | 25 | $P^2HP^2(H^2P^4)^3H^2$ | $-9$ |
| 3d4 | 36 | $P(P^2H^2)^2P^5H^5(H^2P^2)^2P^2H(HP^2)^2$ | $-18$ |
| 3d5 | 46 | $P^2H^3PH^3P^3HPH^2PH^2P^2HPH^4$ $PHP^2H^5PHPH^2P^2H^2P$ | $-35$ |
| 3d6 | 48 | $P^2H(P^2H^2)^2P^5H^{10}P^6$ $(H^2P^2)^2HP^2H^5$ | $-31$ |
| 3d7 | 50 | $H^2(PH)^3PH^4PH(P^3H)^2P^4$ $(HP^3)^2HPH^4(PH)^3PH^2$ | $-34$ |
| 3d8 | 58 | $PH(PH^3)^2P(PH^2PH)^2H(HP)^3$ $(H^2P^2H)^2PHP^4(H(P^2H)^2)^2$ | $-44$ |

where $x_{i,c} = 1$ means residue $i$ occupies position $c$, and $x_{i,c} = 0$ otherwise. The grid dimension $L$ needs to provide enough space for sequences of length $n$, typically satisfying $L \geq \lceil n^{1/3} \rceil + 2$.

For each pair of non-consecutive hydrophobic residues $(i, j)$ with $|i - j| > 1$ and $S_i = S_j = \text{H}$, we introduce contact indicators:

$$c_{i,j} \in \{0, 1\}, \tag{7}$$

where $c_{i,j} = 1$ if residues $i$ and $j$ occupy adjacent cells at Manhattan distance 1, and $c_{i,j} = 0$ otherwise.

### 2.2.2 Core Constraints

The CP-SAT model enforces four main constraint types:

**Placement constraint.** Each residue must sit in exactly one cell:

$$\sum_{c \in \mathcal{G}} x_{i,c} = 1 \quad \forall i \in \{0, \ldots, n-1\}. \tag{8}$$

**Self-avoidance constraint.** Each cell can hold at most one residue:

$$\sum_{i=0}^{n-1} x_{i,c} \leq 1 \quad \forall c \in \mathcal{G}. \tag{9}$$

**Adjacency constraint.** Consecutive residues in the sequence must occupy neighbouring cells on the lattice, creating the self-avoiding walk. For each consecutive pair $(i, i+1)$, if residue $i$ sits in cell $c$, then residue $i+1$ must occupy one of the six adjacent cells:

$$\mathcal{N}(c) = \{c' \in \mathcal{G} : \|c - c'\|_1 = 1\}. \tag{10}$$

We encode this using logical implications:

$$x_{i,c} = 1 \implies \sum_{c' \in \mathcal{N}(c)} x_{i+1,c'} = 1 \quad \forall i \in \{0, \ldots, n-2\}, \forall c \in \mathcal{G}. \tag{11}$$

**Contact linking constraint.** The contact indicators $c_{i,j}$ are linked to placement variables through reified constraints (constraints represented as boolean variables that can be true or false) that ensure $c_{i,j} = 1$ exactly when residues $i$ and $j$ occupy adjacent cells, while excluding consecutive backbone edges.

### 2.2.3 Symmetry Breaking

The cubic lattice has 48 symmetries from rotations and reflections. To cut down the search space and speed up solving, we fix the first two residues at the grid center:

$$x_{0,(\lfloor L/2 \rfloor, \lfloor L/2 \rfloor, \lfloor L/2 \rfloor)} = 1, \tag{12}$$

$$x_{1,(\lfloor L/2 \rfloor, \lfloor L/2 \rfloor, \lfloor L/2 \rfloor - 1)} = 1. \tag{13}$$

This pins residue 0 at the center and residue 1 along the negative $z$-axis, which canonicalises the representation by providing a unique representative conformation from each equivalence class of symmetry-related solutions. This approach shrinks the search space by a factor of 48 without losing any optimal solutions.

#### 2.2.4 Objective Function

The goal is to maximise the total number of non-consecutive H-H contacts. Let $\mathcal{H} = \{i : s_i = \text{H}\}$ denote the set of hydrophobic residue indices. The objective is:

$$\text{maximise} \quad \sum_{(i,j)\in\mathcal{C}} c_{i,j}, \tag{14}$$

where $\mathcal{C} = \{(i,j) : i,j \in \mathcal{H},\, i < j,\, |i-j| > 1\}$ is the set of potential non-consecutive H-H contact pairs. Since the energy function is $E(C) = -\varepsilon_{HH} \cdot \sum_{(i,j)\in\mathcal{C}} c_{i,j}$, maximising contacts is equivalent to minimising energy.

#### 2.2.5 Solver Configuration

We use Google OR-Tools CP-SAT version 9.11 with this setup for each sequence:

- **Optimisation phase:** 560 seconds (20% of time budget) for finding the best grid size and tuning parameters across 5 random seeds
- **Final solve phase:** 2240 seconds (80% of time budget) with optimised settings
- **Parallelisation:** 8 worker threads running parallel search
- **Total time budget:** 2800 seconds (roughly 47 minutes per sequence)

We determine grid size $L$ during the optimisation phase by testing values from $\lceil n^{1/3} \rceil + 2$ up to this value plus 3, picking the smallest grid that gives good solutions. The solver uses branch-and-bound search (a divide-and-conquer method that recursively splits the solution space while pruning branches proven suboptimal) with constraint propagation, conflict-driven clause learning, and automatic search strategies. When the solver finishes with `OPTIMAL` status, the solution is provably the best possible. When it stops with `FEASIBLE` status, the solution is valid but we can't guarantee it's optimal.

### 2.3 LSTM-A Deep Q-Learning Approach

This section describes the LSTM-A (Long Short-Term Memory with multi-head Attention) architecture from Espitia et al. (2024) [17], a deep reinforcement learning approach that combines recurrent neural networks with attention mechanisms for protein folding. The method uses Deep Q-Learning to train an agent that learns to place residues sequentially on the 3D lattice, maximising H-H contacts through trial-and-error interaction with the environment. Unlike purely feedforward or transformer-based Q-networks, LSTM-A processes the folding trajectory autoregressively using LSTM hidden states to maintain memory of the partial conformation, while multi-head attention layers enable the model to attend to all previously placed residues when selecting the next placement action. We use their publicly available implementation, training models from scratch on our local machine for each benchmark sequence.

#### 2.3.1 Markov Decision Process Formulation

The 3D HP folding problem is formulated as a Markov Decision Process (MDP) defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$.

The **state space** $\mathcal{S}$ consists of one-hot encoded vectors representing the folding trajectory. Each state is represented as a tensor of shape $(n, 8, 1)$, where $n$ is the sequence length. At each position in the sequence, the 8-dimensional vector encodes:

$$s_i = [\text{ND}, \text{F}, \text{L}, \text{R}, \text{U}, \text{D}, \text{H}, \text{P}], \tag{15}$$

where the first six elements represent the available actions (No Decision, Forward, Left, Right, Up, Down) and the last two elements indicate the amino acid type (Hydrophobic or Polar). Importantly, the network does not directly observe 3D coordinates; instead, it infers spatial configurations from the sequence of actions taken.

The **action space** $\mathcal{A}$ consists of five discrete relative placement directions:

$$\mathcal{A} = \{\text{Forward}, \text{Left}, \text{Right}, \text{Up}, \text{Down}\}. \tag{16}$$

Each action specifies where to place the next residue relative to the current chain orientation. Backward moves are excluded since backtracking would violate the self-avoiding walk constraint.

The **transition dynamics** are deterministic:

$$P(s_{t+1}|s_t, a_t) : \text{deterministic}, \tag{17}$$

where action $a_t$ determines the placement of the next residue based on the current chain orientation. Episodes terminate successfully when all $n$ residues are placed without collision, or terminate early if the agent reaches a configuration where no valid placements remain.

The **reward function** $R(s_t, a_t)$ is sparse, providing feedback only upon episode termination:

$$R(s_t, a_t) = \begin{cases} \sum_{(i,j)} c_{i,j} & \text{if all residues placed successfully} \\ 0 & \text{otherwise} \end{cases}, \tag{18}$$

where $c_{i,j} = 1$ indicates a valid H-H contact between non-consecutive hydrophobic residues $i$ and $j$ at Manhattan distance 1. This reward equals the negative energy $-E(C)$ from the problem formulation.

### 2.3.2 Environment Implementation

The implementation by Espitia et al. uses a custom Gymnasium environment with the following features:

**Bounded coordinate system.** Residue positions are constrained to lie within a cubic bounding box:

$$[-r, r]^3 \quad \text{where} \quad r = \lfloor n/2 \rfloor. \tag{19}$$

This prevents unbounded exploration and focuses search on compact conformations.

**Symmetry breaking.** To reduce the search space by a factor of 48 (24 rotational symmetries $\times$ 2 reflections), the environment enforces two constraints: the first non-forward turn must be to the right, and the first vertical deviation must be upward. This canonicalises equivalent conformations under lattice symmetries, providing a unique representative from each symmetry class.

**Trap detection.** The environment identifies dead-end configurations where no valid actions remain for the next residue placement. When trapped, the episode terminates immediately with zero reward, providing an implicit penalty that discourages exploration of unproductive regions.

**Action masking.** At each step, the environment computes a boolean mask:

$$m \in \{0, 1\}^{|\mathcal{A}|}, \tag{20}$$

indicating which actions satisfy three criteria: they do not violate the bounded region, they do not cause collisions with existing residues, and they do not lead immediately to a trapped state. This mask is used during action selection to restrict the agent's choices to feasible actions only.

### 2.3.3 LSTM-A Network Architecture

The action-value function $Q(s, a)$ uses the LSTM-A architecture from Espitia et al. [17], which combines Long Short-Term Memory networks with multi-head attention. The architecture processes the $(n, 8, 1)$ state tensor representing one-hot encoded actions and amino acid types:

**LSTM layer.** The input tensor is processed through an LSTM layer with hidden size $d = 512$. The LSTM maintains hidden states that capture sequential dependencies in the folding trajectory, processing each timestep's 8-dimensional one-hot vector sequentially.

**Multi-head attention.** The LSTM outputs at all positions are refined using a multi-head self-attention mechanism with 4 heads. Given the LSTM hidden states $H \in \mathbb{R}^{n \times 512}$, each attention head computes:

$$\text{Attention}_i(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \tag{21}$$

where $Q$, $K$, and $V$ are linear projections of the LSTM output $H$, and $d_k$ is the dimension of the key vectors. The softmax function converts a vector of real numbers into a probability distribution:

$$\text{softmax}(\mathbf{z})_j = \frac{\exp(z_j)}{\sum_{k=1}^{n} \exp(z_k)}, \tag{22}$$

where $\mathbf{z} = [z_1, \ldots, z_n]$ is the input vector. The output is a vector of positive values that sum to 1, which ensures attention weights form a valid probability distribution. The outputs from all heads are concatenated and projected:

$$\text{MultiHead}(H) = \text{Concat}(\text{head}_1, \ldots, \text{head}_4)W^O. \tag{23}$$

This allows the model to attend to all previously placed residues when evaluating the next action, enabling it to capture both local sequential patterns and long-range dependencies in the protein structure.

**Output layer.** The final output is obtained by selecting the last timestep of the attention output, followed by a fully connected layer that maps to Q-values for the 5 placement actions.

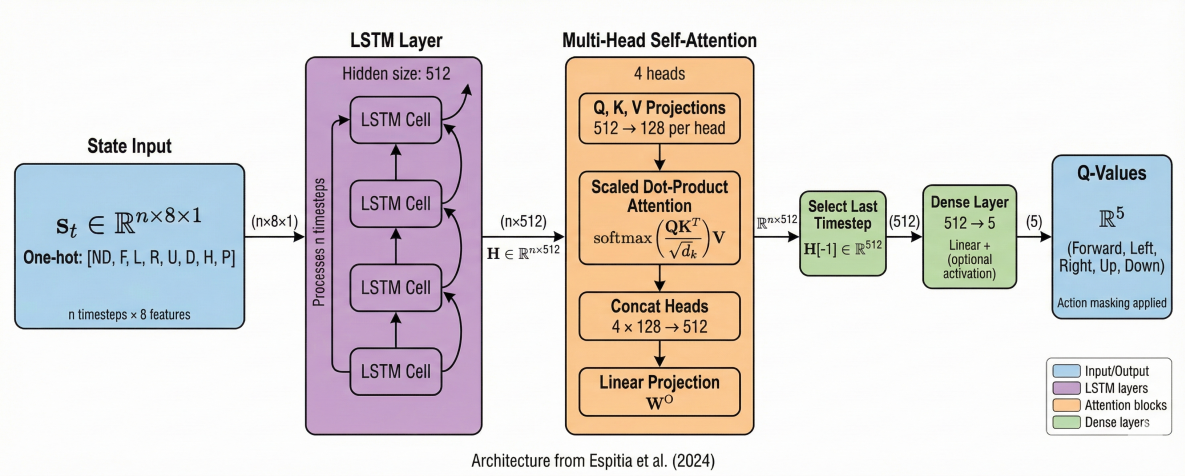



Figure 2: LSTM-A architecture from Espitia et al. (2024). The state input $(n, 8, 1)$ is processed through LSTM cells (hidden size 512) followed by 4-head multi-head self-attention. The last timestep of the attention output is mapped to Q-values for the 5 placement actions through a dense layer. Figure created by the author using Nano Banana.

### 2.3.4 Training Protocol

The training procedure in Espitia et al. uses Deep Q-Learning with several stabilisation techniques:

**Double DQN.** To mitigate overestimation bias, two networks are maintained: a policy network $Q(s, a; \theta)$ and a target network $Q(s, a; \theta^-)$. The policy network selects actions during target computation, while the target network evaluates them:

$$y_t = r_t + \gamma\, Q(s_{t+1}, \arg\max_{a'} Q(s_{t+1}, a'; \theta); \theta^-). \tag{24}$$

The target network parameters $\theta^-$ are updated by copying from the policy network every 1000 training steps.

**Prioritised experience replay.** Transitions $(s_t, a_t, r_t, s_{t+1}, d_t)$ are stored in a replay buffer. Transitions are sampled with probability proportional to their temporal-difference error:

$$p_t \propto |\delta_t|^\alpha \quad \text{where} \tag{25}$$

$$\delta_t = y_t - Q(s_t, a_t; \theta) \quad \text{and} \quad \alpha = 0.6, \tag{26}$$

where $\delta_t$ is the TD error and $\alpha$ controls prioritisation strength. Since high-priority transitions are sampled more frequently, they would be overrepresented in gradient updates without correction. Importance sampling weights address this bias:

$$w_t = (N \cdot p_t)^{-\beta}, \tag{27}$$

where $N$ is the replay buffer size, $p_t$ is the sampling probability of transition $t$, and $\beta$ anneals from 0.4 to 1.0 during training to gradually increase the bias correction.

**Action masking with $\varepsilon$-greedy exploration.** During training, actions are selected using an $\varepsilon$-greedy policy restricted to valid actions. With probability $\varepsilon$, the agent selects uniformly from the set of valid actions indicated by the environment's action mask. Otherwise, it selects:

$$\arg\max_{a \in \mathcal{A}_{\text{valid}}} Q(s, a; \theta), \tag{28}$$

where invalid actions have their Q-values set to $-10^9$ to prevent selection. The exploration rate decays exponentially:

$$\varepsilon_t = \max(0.01, 1.0 \cdot 0.9995^t). \tag{29}$$

**Hyperparameters and training details.** Following Espitia et al., separate models are trained for each benchmark sequence. Table 2 summarises the training configuration.

Table 2: LSTM-A training hyperparameters from Espitia et al. (2024)

| **Parameter** | **Value** |
|---|---|
| *Network Architecture* | |
| LSTM hidden size | 512 |
| Attention heads | 4 |
| *Training Configuration* | |
| Optimiser | Adam |
| Learning rate | $1.0 \times 10^{-3}$ |
| Discount factor $\gamma$ | 0.98 |
| Batch size | 16 (short seq.), 32 (long seq.) |
| Training episodes | 200K–250K (short), 500K–750K (long) |
| *Stabilisation Techniques* | |
| Target network update | Every 1,000 steps |
| PER $\alpha$ (prioritisation) | 0.6 |
| PER $\beta$ (importance sampling) | $0.4 \rightarrow 1.0$ (annealed) |
| $\varepsilon$-greedy decay | $\max(0.01, 1.0 \cdot 0.9995^t)$ |

## 3 Experiments and Results

### 3.1 Experimental Setup

All experiments were conducted on an Apple MacBook Pro with M2 Max chip and 32 GB unified memory. The CP-SAT solver was implemented using Google OR-Tools version 9.11, while the LSTM-A DQN implementation was obtained from Espitia et al. (2024) [17] and executed locally. Both methods were evaluated on the standard 3D HP benchmark sequences (3d1–3d8) published by [17], with best-known energy values serving as optimality targets.

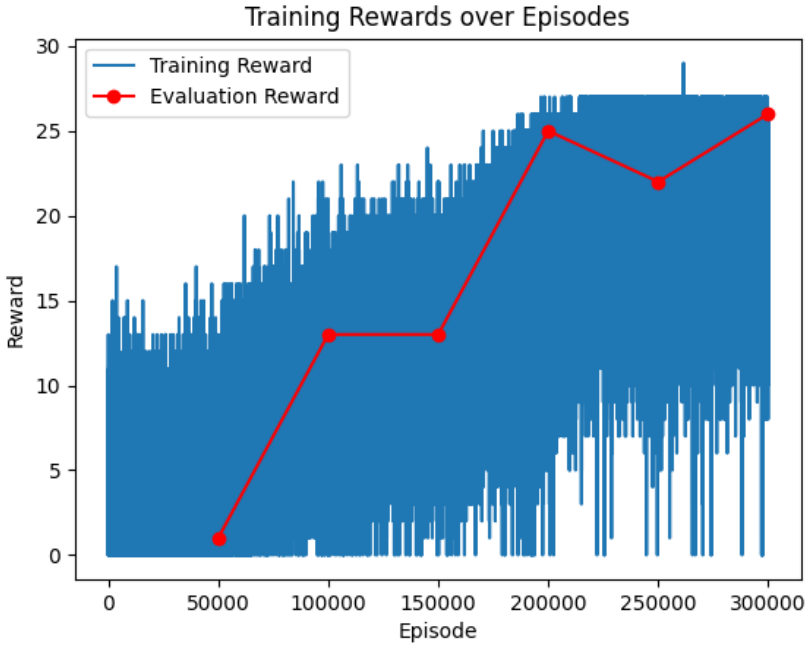
### 3.2 CP-SAT Experiments

For CP-SAT, we allocated a total computational budget of 7 hours distributed across all 8 sequences (3d1–3d8), yielding approximately 47 minutes per sequence. Each sequence underwent a two-phase optimisation: 20% of the budget (9 minutes, 20 seconds) for grid size exploration and parameter tuning across 5 random seeds, and 80% of the budget (37 minutes, 20 seconds) for final optimisation with the best-discovered settings. The solver used 8 parallel worker threads throughout. All CP-SAT results reported in this chapter were obtained from our own experimental runs.

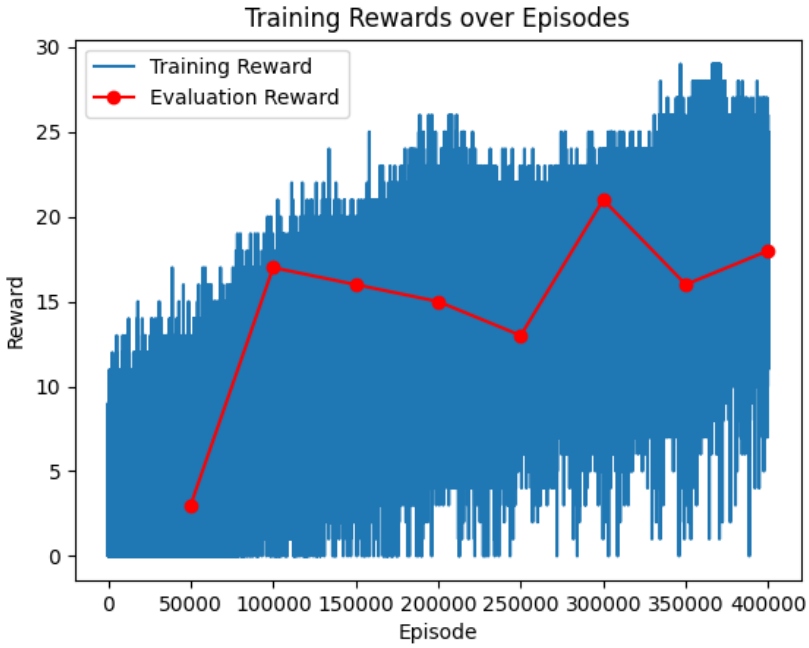
### 3.3 LSTM-A DQN Experiments

For the LSTM-A agent, we attempted to replicate the training protocol described in Section 2.3 on our local machine. However, due to the substantial computational requirements and extended training times on CPU (the M2 Max lacks CUDA support for PyTorch), we were only able to complete training runs for sequences 3d1 and 3d2.

Given these resource constraints, we report a combination of results: our locally obtained results for 3d1 and 3d2, supplemented by the published results from Espitia et al. (2024) for sequences 3d3–3d8. Figure 3 shows representative training dynamics from Espitia et al. for sequences of varying lengths, illustrating how training complexity scales with sequence size.




(a) Length 48 (3d6)




(b) Length 50 (3d7)

Figure 3: LSTM-A training convergence curves from Espitia et al. (2024) for sequences of varying lengths. Blue areas show training rewards, red lines show evaluation performance.

All results tables in this chapter clearly indicate the source of each result, distinguishing between our experimental runs and values taken from the original paper.

### 3.4 Solution Quality and Comparative Analysis

Table 3 presents the energy values achieved by both methods on the 3D HP benchmark sequences. The energy is defined as the negative count of non-consecutive H-H contacts, so more negative values indicate better (more stable) conformations.

Table 3: Solution quality on 3D HP benchmark sequences. Energy is the negative count of H-H contacts; lower (more negative) values are better. All CP-SAT results from our experiments; LSTM-A results: †our local runs, others from Espitia et al. (2024).

| Seq. | Len | CP-SAT | LSTM-A | Best Known |
|---|---|---|---|---|
| 3d1 | 20 | $-11$ | $-11^{\dagger}$ | $-11$ |
| 3d2 | 24 | $-13$ | $-13^{\dagger}$ | $-13$ |
| 3d3 | 25 | $-9$ | $-9$ | $-9$ |
| 3d4 | 36 | $-18$ | $-18$ | $-18$ |
| 3d5 | 46 | $-32$ | $-33$ | $-35$ |
| 3d6 | 48 | $-29$ | $-30$ | $-31$ |
| 3d7 | 50 | $-30$ | $-29$ | $-34$ |
| 3d8 | 58 | $-40$ | $-40$ | $-44$ |

Both methods demonstrate strong performance on shorter sequences (3d1–3d4, lengths 20–36), matching the best-known energy values exactly. The CP-SAT solver terminated with `OPTIMAL` status for these sequences, providing certificates of optimality through exhaustive branch-and-bound search within the allocated time budget. Based on our local runs for 3d1–3d2 and published results from Espitia et al. for 3d3–3d8, the LSTM-A agent matched these optimal values through learned sequential placement policies, successfully recognising and exploiting hydrophobic clustering patterns via its LSTM-attention architecture.

On longer sequences (3d5–3d8, lengths 46–58), neither method reaches the best-known values. CP-SAT maintains gaps of 2–4 contacts from the literature values, unable to prove optimality within the time limit as the exponential search space growth overwhelms the constraint propagation and symmetry-breaking techniques. LSTM-A shows similar performance with gaps of 2–5 contacts: achieving $-33$ on 3d5 (within 2 of best-known), $-30$ on 3d6 (within 1), $-29$ on 3d7 (5 away), and $-40$ on 3d8. Notably, both methods converge to identical energy values on the longest sequence 3d8 (length 58), suggesting they face similar fundamental challenges in scaling to longer sequences.

Figure 4 visualises these results, clearly showing the transition from optimal performance on shorter sequences to increasing gaps on longer ones.

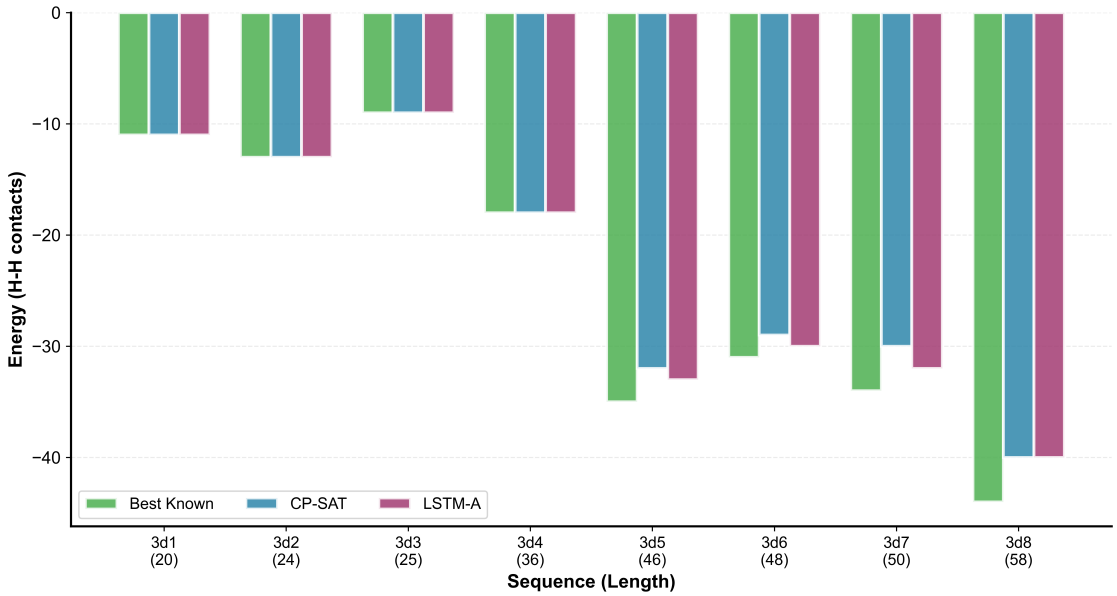



Figure 4: Solution quality comparison across benchmark sequences. Both methods achieve optimal solutions for sequences up to length 36 (3d1–3d4). Performance degrades gracefully on longer sequences, with gaps of 2–5 contacts from best-known values.

Figure 5 illustrates representative conformations for sequence 3d7 (length 50), where CP-SAT achieved $-30$ contacts compared to LSTM-A's $-29$ contacts, demonstrating the one-contact advantage of systematic constraint-based search on this instance.

(a) CP-SAT solution: Energy = $-30$ contacts

(b) LSTM-A solution: Energy = $-29$ contacts

Figure 5: Conformations for sequence 3d7 (length 50). Panel a: CP-SAT achieves $-30$ contacts (red = H, blue = P). Panel b: LSTM-A achieves $-29$ contacts (orange = H, cyan = P). Magenta lines mark non-consecutive H-H contacts.

## 4 Discussion

This study compared constraint programming (CP-SAT) against reinforcement learning (LSTM-A) for the 3D HP protein folding problem, addressing whether systematic search with guarantees or learning-based exploration better handles this NP-hard combinatorial optimisation task.

Both methods achieved identical results on sequences up to length 36, with CP-SAT providing optimality certificates through exhaustive search and LSTM-A discovering the same conformations through learned policies. This demonstrates that reinforcement learning can match exact optimisation quality when the problem remains tractable. The LSTM-attention architecture successfully captured geometric patterns for maximising hydrophobic contacts, validating that neural networks can learn the structure of protein conformational landscapes. Performance diverged on longer sequences (46–58 residues), where both methods fell 2–5 contacts short of best-known values. CP-SAT's gaps reflect exponential

search space growth overwhelming constraint propagation, while LSTM-A's gaps suggest learned policies struggle to generalise beyond training distributions. Their convergence to identical energies on the longest sequence (3d8: both achieving $-40$) indicates they encounter similar local optima despite completely different search strategies.

CP-SAT provides optimality guarantees when tractable, making it ideal for benchmarking and establishing ground truth. The solver's deterministic behaviour ensures reproducibility and enables rigorous analysis through statistics. However, NP-hardness fundamentally limits scalability. Even sophisticated pruning eventually fails as search space grows exponentially. LSTM-A learns generalisable placement policies from experience, potentially enabling faster solutions on similar sequences through transfer learning. The architecture naturally models both sequential dependencies (LSTM) and long-range relationships (attention). Our replication of published 3d1–3d2 results confirms reproducibility, though computational constraints prevented validation on longer sequences.

Computational constraints limited our LSTM-A validation to sequences 3d1–3d2, with longer sequences using published results from Espitia et al. While successful replication confirms reproducibility, independent verification would strengthen conclusions. Fixed CP-SAT time budgets (47 minutes per sequence) may not reflect true asymptotic performance, and we adopted published LSTM-A hyperparameters without extensive per-sequence tuning.

Future work should explore hybrid approaches combining CP-SAT's optimality-seeking with LSTM-A's pattern recognition. For example, using CP-SAT to generate diverse training data or learned policies to warm-start branch-and-bound search. Transfer learning deserves investigation: can models pretrained on shorter sequences accelerate longer ones? Do models transfer across sequences with similar hydrophobic patterns?

## Code Availability

The software, code, and data used in this study are available in public GitHub repositories and Github Gists:

- **CP-SAT Implementation:**
  `https://github.com/pranav-ramanathan/HP-CSP`
- **LSTM-A DQN Implementation:**
  `https://github.com/AnonymousAuthor117/Protein-Structure-Prediction-in-the-3D-HP-Model-Using-Deep-Reinforcement-Learning`
- **Image Generation Prompts:**
  `https://gist.github.com/pranav-ramanathan/f5a4f8effa07861ef452e802132e2dc8`

## References

[1] Cyrus Levinthal. How to fold graciously. In *Mossbauer Spectroscopy in Biological Systems: Proceedings of a meeting held at Allerton House, Monticello, Illinois*, pages 22–24, 1969.

[2] National Center for Biotechnology Information. Hemoglobin subunit beta. `https://www.ncbi.nlm.nih.gov/gene/3043`, 2025. Accessed: 2025.

[3] National Center for Biotechnology Information. Biochemistry, essential amino acids. `https://www.ncbi.nlm.nih.gov/books/NBK557845/`, 2025. Accessed: 2025.

[4] William A. Eaton, Victor Muñoz, Paul A. Thompson, Cheng-Yen Chan, and James Hofrichter. Submillisecond kinetics of protein folding. *Current Opinion in Structural Biology*, 6(1):110–116, 1996.

[5] Ken A. Dill. Theory for the folding and stability of globular proteins. *Biochemistry*, 24(6):1501–1509, 1985.

[6] Kit Fun Lau and Ken A. Dill. A lattice statistical mechanics model of the conformational and sequence spaces of proteins. *Macromolecules*, 22(10):3986–3997, 1989.

[7] Pierluigi Crescenzi, Deborah Goldman, Christos Papadimitriou, Antonio Piccolboni, and Mihalis Yannakakis. On the complexity of protein folding. *Journal of Computational Biology*, 5(3):423–465, 1998.

[8] Ron Unger and John Moult. Genetic algorithms for protein folding simulations. *Journal of Molecular Biology*, 231(1):75–81, 1993.

[9] Natalio Krasnogor, B. P. Blackburne, Edmund K. Burke, and Jonathan D. Hirst. Multimeme algorithms for protein structure prediction. In Juan Julián Merelo Guervós, Panagiotis Adamidis, Hans-Georg Beyer, Hans-Paul Schwefel, and José Luis Fernández-Villacañas, editors, *Parallel Problem Solving from Nature – PPSN VII*, volume 2439 of *Lecture Notes in Computer Science*, pages 769–778. Springer, Berlin, Heidelberg, 2002. doi:10.1007/3-540-45712-7_74. URL `https://dl.acm.org/doi/10.5555/645826.669429`.

[10] Neal Lesh, Michael Mitzenmacher, and Sue Whitesides. A complete and effective move set for simplified protein folding. In *Proceedings of the 7th Annual International Conference on Computational Molecular Biology (RECOMB'03)*, pages 188–195, 2003.

[11] Thomas Stützle and Holger H. Hoos. Max-min ant system. *Future Generation Computer Systems*, 16(8):889–914, 2000.

[12] Roman Barták. Theory and practice of constraint propagation. In *Proceedings of the 3rd Workshop on Constraint Programming for Decision and Control*, pages 7–14, 2001.

[13] Martin Mann, Sebastian Will, and Rolf Backofen. Cpsp-web-tools: a server for 3d lattice protein studies. *Bioinformatics*, 25(5):676–677, 2009.

[14] Laurent Perron, Frédéric Didier, and Vianney Furnon. The cp-sat-lp solver. In *29th International Conference on Principles and Practice of Constraint Programming (CP 2023)*, 2023.

[15] Google. Or-tools: Google optimization tools. `https://developers.google.com/optimization`, 2024.

[16] Paul Shaw. Using constraint programming and local search methods to solve vehicle routing problems. In *Principles and Practice of Constraint Programming*, pages 417–431, 1998.

[17] Gabriel Espitia, Yuanqi T. Pang, and James C. Gumbart. Protein structure prediction in the 3d hp model using deep reinforcement learning. *arXiv preprint arXiv:2412.20329*, 2024.

[18] Kaiming Yang et al. Applying deep reinforcement learning to the hp model for protein structure prediction. *Physica A: Statistical Mechanics and Its Applications*, 608:128395, 2022.